Modul 7 — Autentikasi dan Otorisasi Mini Project Sekolah

> **Prasyarat:** Modul 6 sudah berjalan.
> **Lingkungan:** PHP 8.0+, PDO MySQL, session, browser, XAMPP.
> **Alokasi:** 6 pertemuan × 3 JP.
> **Hasil:** login, logout, hashing password, session timeout, role, serta proteksi panel baru dan halaman lama.

> Materi ini adalah implementasi latihan lokal, bukan sertifikasi bahwa aplikasi siap produksi. Sebelum dipublikasikan diperlukan HTTPS, rate limiting yang memadai, konfigurasi server, pengelolaan rahasia, audit, backup, dan pemantauan. Gunakan akun serta data latihan, bukan data pribadi siswa sebenarnya.

1. Tujuan pembelajaran

1. Membedakan autentikasi dan otorisasi.
2. Menyimpan hash password, bukan password asli atau enkripsi yang dapat dibalik.
3. Menggunakan session server-side dengan cookie yang sesuai.
4. Mengganti ID session setelah login dan membersihkan session saat logout.
5. Memeriksa role di server pada setiap jalur perubahan data.
6. Menerapkan CSRF pada login, logout, dan operasi CRUD.
7. Menguji akses langsung URL, session kedaluwarsa, akun nonaktif, dan role berubah.

2. Konsep dan kebijakan akses

Autentikasi: “Siapa pengguna ini?” Dibuktikan dengan username dan password.

Otorisasi: “Apa yang boleh dilakukan pengguna ini?” Diputuskan server berdasarkan role dan kebijakan.

Session: data di server yang dihubungkan ke browser melalui ID session pada cookie. Cookie bukan tempat menyimpan password atau role yang dipercaya dari pengguna.

Aksi	Belum login	viewer	operator	admin
Login	Boleh	Sudah login	Sudah login	Sudah login
Daftar dan rapor	Redirect login	Boleh	Boleh	Boleh
Form tambah/edit	Redirect login	403	Boleh	Boleh
Simpan/hapus data	Redirect login	403	Boleh	Boleh
Logout POST	Tidak merusak data	Boleh	Boleh	Boleh

Aksi	Belum login	viewer	operator	admin
Kelola akun lewat web	Tidak tersedia	Tidak tersedia	Tidak tersedia	Tidak tersedia

Pembuatan akun pada modul ini hanya melalui CLI oleh pengelola server. Tidak ada registrasi publik, sehingga pengguna tidak dapat memilih role admin sendiri. Role admin disediakan untuk pengembangan manajemen akun di masa depan; pada CRUD saat ini hak tulisnya sama dengan operator.

Menyembunyikan tombol bukan kontrol akses. Pengguna tetap dapat mengirim request langsung, sehingga endpoint harus selalu memeriksa izin.

3. Struktur file tambahan

```
crud_siswa\
├── koneksi.php
├── auth.php                ← fungsi session, login guard, role
├── login.php               ← halaman publik khusus login
├── logout.php              ← hanya POST + CSRF
├── buat_akun.php            ← hanya CLI, menolak request web
├── index.php               ← halaman lama: ditambah guard
├── detail.php              ← halaman lama: ditambah guard
├── tambah.php              ← guard role + CSRF
├── edit.php                ← guard role + CSRF
├── hapus.php               ← guard role + CSRF
└── admin\
    ├── bootstrap.php        ← diganti dengan versi modul ini
    ├── helper.php           ← tetap dari Modul 6
    ├── index.php            ← ditambah pemeriksaan role
    └── ...                 ← repository/validator tetap
```

`login.php` **tidak** memuat `admin/bootstrap.php`, karena bootstrap meminta pengguna login. Jika login memuat guard itu, akan terjadi redirect berulang.

4. Tabel pengguna

Jalankan SQL berikut pada `db_sekolah`. Tabel pengguna terpisah dari siswa: akun operator tidak harus merupakan siswa, dan tidak semua siswa memiliki akun.

```
CREATE TABLE users (
    id_user INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    role ENUM('admin', 'operator', 'viewer') NOT NULL DEFAULT 'viewer',
    aktif TINYINT(1) NOT NULL DEFAULT 1,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Jika tabel sudah ada, periksa `SHOW CREATE TABLE users` terlebih dahulu; jangan menghapusnya untuk mengulang tutorial. Panjang 255 memberi ruang bagi perubahan algoritma `PASSWORD_DEFAULT` di masa depan.

Username latihan dibatasi huruf kecil ASCII, angka, dan underscore sepanjang 3–50 karakter. Normalisasi yang sama digunakan saat membuat akun dan login. Nama tampilan dapat ditambahkan sebagai kolom terpisah jika diperlukan.

5. File lengkap `auth.php`

```
<?php
declare(strict_types=1);

require_once __DIR__ . '/koneksi.php';
require_once __DIR__ . '/admin/helper.php';

const APP_BASE = '/crud_siswa';
const AUTH_IDLE_SECONDS = 1800;
const AUTH_ABSOLUTE_SECONDS = 28800;

if (session_status() !== PHP_SESSION_ACTIVE) {
    ini_set('session.use_strict_mode', '1');
    ini_set('session.use_only_cookies', '1');
    $https = isset($_SERVER['HTTPS']) && $_SERVER['HTTPS'] !== 'off'
        && $_SERVER['HTTPS'] !== '';
    session_name('sekolah_session');
    session_set_cookie_params([
        'lifetime' => 0,
        'path' => APP_BASE . '/',
        'secure' => $https,
        'httponly' => true,
        'samesite' => 'Lax',
    ]);
    session_start();
}

$pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
$pdo->setAttribute(PDO::ATTR_DEFAULT_FETCH_MODE, PDO::FETCH_ASSOC);
$pdo->setAttribute(PDO::ATTR_EMULATE_PREPARES, false);

function hapusIdentitas(): void
{
    $_SESSION = [];
    session_regenerate_id(true);
}

function currentUser(): ?array
{
    global $pdo;
    $id = $_SESSION['user_id'] ?? null;
```

```
    if (!is_int($id) || $id < 1) {
        return null;
    }
    $sekarang = time();
    $login = $_SESSION['login_at'] ?? 0;
    $aktif = $_SESSION['last_seen'] ?? 0;
    if (!is_int($login) || !is_int($aktif)
        || $sekarang - $aktif >= AUTH_IDLE_SECONDS
        || $sekarang - $login >= AUTH_ABSOLUTE_SECONDS) {
        hapusIdentitas();
        return null;
    }
    $stmt = $pdo->prepare('SELECT id_user, username, role, aktif FROM users WHERE id_user = :id');
    $stmt->execute(['id' => $id]);
    $user = $stmt->fetch();
    if (!$user || (int) $user['aktif'] !== 1
        || !in_array($user['role'], ['admin', 'operator', 'viewer'], true)) {
        hapusIdentitas();
        return null;
    }
    $_SESSION['last_seen'] = $sekarang;
    return $user;
}

function requireLogin(): array
{
    $user = currentUser();
    if ($user === null) {
        header('Location: ' . APP_BASE . '/login.php', true, 303);
        exit;
    }
    header('Cache-Control: no-store');
    return $user;
}

function requireRole(array $roles): array
{
    $user = requireLogin();
    if (!in_array($user['role'], $roles, true)) {
        http_response_code(403);
        exit('Akses ditolak. Akun ini tidak memiliki izin perubahan data.');
    }
    return $user;
}
```

5.1 Penjelasan pilihan desain

- `APP_BASE` harus disesuaikan jika nama folder berbeda. Ini konstanta yang ditentukan pengembang, bukan nilai dari request.

- Cookie `HttpOnly` mencegah JavaScript membaca cookie melalui API biasa, tetapi tidak menghilangkan seluruh dampak XSS.
- `SameSite=Lax` menambah perlindungan, tetapi tidak menggantikan token CSRF.
- `Secure` aktif ketika PHP mengetahui koneksi HTTPS. Pada localhost HTTP nilainya false agar cookie dapat digunakan. Produksi harus memaksa HTTPS; jangan mempercayai header proxy dari sembarang klien.
- `session.use_strict_mode` menolak ID session yang tidak dikenal oleh penyimpanan session.
- Idle timeout 30 menit dihitung server-side; absolute timeout 8 jam tidak diperpanjang oleh aktivitas.
- Role dan status aktif dibaca dari database setiap request. Menonaktifkan akun atau mengganti role berlaku pada request berikutnya tanpa harus menunggu login ulang.
- Password tidak disimpan di session; session hanya menyimpan ID pengguna dan waktu.
- Penggunaan `global $pdo` dibatasi sebagai kompromi tutorial kecil. Refactoring lanjutan dapat memindahkan fungsi ke `AuthService` yang menerima PDO melalui constructor.

Fungsi di atas tidak menelan kegagalan database. Jika database gagal, request gagal, bukan otomatis dianggap terautentikasi. Pada server produksi gunakan penanganan error terpusat dan `display_errors=Off` agar detail teknis tidak ditampilkan.

6. Membuat akun pertama melalui CLI

Simpan file berikut sebagai `buat_akun.php`. Jalankan hanya pada terminal pengelola lokal. Password dibaca interaktif, bukan melalui argumen command line agar tidak masuk riwayat perintah. Pada terminal sederhana input masih dapat terlihat; hindari berbagi layar dan gunakan terminal privat.

```
<?php
declare(strict_types=1);

if (PHP_SAPI !== 'cli') {
    http_response_code(404);
    exit;
}
require_once __DIR__ . '/koneksi.php';
$pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

function bacaTerminal(string $label): string
{
    fwrite(STDOUT, $label);
    $baris = fgets(STDIN);
    if ($baris === false) {
        throw new RuntimeException('Input terminal tidak tersedia.');
    }
    return rtrim($baris, "\r\n");
}

try {
    $username = strtolower(trim(bacaTerminal('Username (huruf kecil/angka/underscore): ')));
    $role = trim(bacaTerminal('Role (admin/operator/viewer): '));
    $password = bacaTerminal('Password baru (input terlihat): ');
    $ulang = bacaTerminal('Ulangi password: ');
```

```
    if (!preg_match('/^[a-z0-9_]{3,50}$/D', $username)) {
        throw new InvalidArgumentException('Username tidak memenuhi aturan.');
    }
    if (!in_array($role, ['admin', 'operator', 'viewer'], true)) {
        throw new InvalidArgumentException('Role tidak valid.');
    }
    if (strlen($password) < 12 || strlen($password) > 72 || str_contains($password, "\0")) {
        throw new InvalidArgumentException('Password harus 12–72 byte dan tidak boleh mengandung NUL.');
    }
    if ($password !== $ulang) {
        throw new InvalidArgumentException('Konfirmasi password tidak sama.');
    }
    $stmt = $pdo->prepare(
        'INSERT INTO users (username, password_hash, role) VALUES (:username, :hash, :role)'
    );
    $stmt->execute([
        'username' => $username,
        'hash' => password_hash($password, PASSWORD_DEFAULT),
        'role' => $role,
    ]);
    unset($password, $ulang);
    echo 'Akun berhasil dibuat.' . PHP_EOL;
} catch (InvalidArgumentException | RuntimeException $e) {
    if ($e instanceof PDOException) {
        echo 'Akun gagal dibuat. Periksa username duplikat atau koneksi database.' . PHP_EOL;
    } else {
        echo $e->getMessage() . PHP_EOL;
    }
    exit(1);
}
```

Jalankan:

```
C:\xampp\php\php.exe C:\xampp\htdocs\crud_siswa\buat_akun.php
```

Buat masing-masing satu akun admin, operator, dan viewer untuk pengujian. Jangan menuliskan password latihan ke repository. Batas 72 **byte** dipilih karena `PASSWORD_DEFAULT` pada versi PHP yang dipakai dapat menggunakan bcrypt yang memiliki batas input tersebut; ini bukan batas 72 karakter Unicode.

7. File lengkap `login.php`

```
<?php
declare(strict_types=1);
require_once __DIR__ . '/auth.php';
header('Cache-Control: no-store');

if (!in_array($_SERVER['REQUEST_METHOD'], ['GET', 'POST'], true)) {
    header('Allow: GET, POST');
```

```
    http_response_code(405);
    exit('Metode tidak tersedia.');
}
if (currentUser() !== null) {
    redirectAdmin(APP_BASE . '/admin/index.php');
}

$error = '';
$username = '';
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    verifyCsrf();
    $rawUsername = $_POST['username'] ?? null;
    $password = $_POST['password'] ?? null;
    if (is_string($rawUsername)) {
        $username = strtolower(trim($rawUsername));
    }
    $bentukValid = is_string($rawUsername) && is_string($password)
        && preg_match('/^[a-z0-9_]{3,50}$/D', $username)
        && strlen($password) >= 12 && strlen($password) <= 72
        && !str_contains($password, "\0");

    $user = false;
    if ($bentukValid) {
        $stmt = $pdo->prepare('SELECT * FROM users WHERE username = :username');
        $stmt->execute(['username' => $username]);
        $user = $stmt->fetch();
    }
    $valid = false;
    if ($bentukValid) {
        // Hash dummy menghindari jalur cepat khusus username yang tidak ditemukan.
        $dummy = '$2y$10$92IXUNpkjO0rOQ5byMi.Ye4oKoEa3Ro9llC/.og/at2uheWG/igi.';
        $valid = password_verify($password, $user ? $user['password_hash'] : $dummy);
    }
    if (!$user || !$valid || (int) $user['aktif'] !== 1
        || !in_array($user['role'], ['admin', 'operator', 'viewer'], true)) {
        http_response_code(401);
        $error = 'Username atau password salah, atau akun tidak tersedia.';
    } else {
        if (password_needs_rehash($user['password_hash'], PASSWORD_DEFAULT)) {
            $stmt = $pdo->prepare('UPDATE users SET password_hash = :hash WHERE id_user = :id');
            $stmt->execute([
                'hash' => password_hash($password, PASSWORD_DEFAULT),
                'id' => $user['id_user'],
            ]);
        }
        $_SESSION = [];
        session_regenerate_id(true);
        $_SESSION['user_id'] = (int) $user['id_user'];
        $_SESSION['login_at'] = time();
        $_SESSION['last_seen'] = time();
        csrfToken();
```

```
        redirectAdmin(APP_BASE . '/admin/index.php');
    }
    unset($password);
}
?>
<!doctype html>
<html lang="id">
<head><meta charset="utf-8"><meta name="viewport" content="width=device-width, initial-scale=1">
<title>Login data sekolah</title></head>
<body>
<h1>Login data sekolah</h1>
<?php if ($error !== ''): ?><p role="alert"><?= eAdmin($error) ?></p><?php endif; ?>
<form method="post" action="<?= eAdmin(APP_BASE . '/login.php') ?>">
    <input type="hidden" name="csrf_token" value="<?= eAdmin(csrfToken()) ?>">
    <p><label>Username
        <input name="username" value="<?= eAdmin($username) ?>" required maxlength="50"
            autocomplete="username">
    </label></p>
    <p><label>Password
        <input name="password" type="password" required autocomplete="current-password">
    </label></p>
    <button type="submit">Login</button>
</form>
</body>
</html>
```

7.1 Apa yang terjadi saat login?

1. Browser mendapatkan form serta token session.
2. POST diperiksa token dan bentuk inputnya.
3. Query parametrik mencari username yang telah dinormalisasi.
4. `password_verify()` membandingkan password dengan hash.
5. Kegagalan memakai pesan yang sama untuk username tidak ada, password salah, dan akun nonaktif.
6. Jika algoritma/biaya hash berubah, `password_needs_rehash()` memungkinkan pembaruan saat password asli tersedia pada login yang sah.
7. Session lama dibersihkan dan ID baru dibuat.
8. Identitas minimum disimpan; token CSRF baru dibuat.
9. Redirect menuju alamat internal yang tetap.

Hash dummy bukan akun dan bukan password login bawaan. Ia hanya membuat jalur username tidak ditemukan tetap melakukan pekerjaan verifikasi hash. Ini tidak menjamin seluruh respons memiliki waktu identik, dan tidak menggantikan rate limiting.

Password tidak di-`trim()`: spasi dapat menjadi bagian password. Jangan mengisi kembali atribut `value` pada input password ketika login gagal.

7.2 Mengapa tidak membandingkan hash sendiri?

Jangan memakai `md5()`, `sha1()`, atau membandingkan `password_hash($input)` dengan hash tersimpan. `password_hash()` menggunakan salt acak, sehingga hash baru dapat berbeda untuk password yang sama. Salt dan informasi algoritma sudah disertakan pada hasil hash; gunakan `password_verify()`.

8. File lengkap `logout.php`

```
<?php
declare(strict_types=1);
require_once __DIR__ . '/auth.php';
header('Cache-Control: no-store');

if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
    header('Allow: POST');
    http_response_code(405);
    exit('Logout harus menggunakan POST.');
}
verifyCsrf();
$_SESSION = [];
$params = session_get_cookie_params();
setcookie(session_name(), '', [
    'expires' => time() - 42000,
    'path' => $params['path'],
    'domain' => $params['domain'],
    'secure' => $params['secure'],
    'httponly' => $params['httponly'],
    'samesite' => $params['samesite'] ?? 'Lax',
]);
session_destroy();
redirectAdmin(APP_BASE . '/login.php');
```

Menghapus `$_SESSION['user_id']` saja tidak sama dengan menghancurkan session dan menghapus cookie. Setelah logout, request baru ke halaman terlindungi harus meminta login kembali. Tombol Back dapat menampilkan snapshot browser tertentu; pemeriksaan sesungguhnya adalah apakah request baru masih dapat membaca/menulis data.

9. Integrasi ke panel Modul 6

9.1 Ganti seluruh `admin\bootstrap.php`

```
<?php
declare(strict_types=1);

require_once dirname(__DIR__) . '/auth.php';
$penggunaAktif = requireLogin();
```

```
require_once __DIR__ . '/DataRepository.php';
require_once __DIR__ . '/validasi.php';

$config = require __DIR__ . '/entitas.php';
$repo = new DataRepository($pdo, $config);
```

`auth.php` sudah memuat koneksi, helper, dan session. Jangan menjalankan `session_start()` lama sebelum konfigurasi cookie pada `auth.php`.

9.2 Tambahkan pemeriksaan role di controller

Pada `admin/index.php`, ganti baris penanda `AUTH_ROLE_CHECK` dengan:

```
if (in_array($aksi, ['form', 'simpan', 'hapus'], true)) {
    requireRole(['admin', 'operator']);
}
```

Posisinya **setelah validasi allowlist aksi, sebelum pemrosesan POST atau pengambilan form**. Dengan demikian viewer tidak dapat menulis meskipun mengirim request manual. Guard pada bootstrap sudah memeriksa autentikasi seluruh panel.

9.3 Tampilkan identitas dan logout

Tambahkan tepat setelah `<body>` pada panel:

```
<p>Masuk sebagai <?= eAdmin($penggunaAktif['username']) ?>
    (<?= eAdmin($penggunaAktif['role']) ?>)</p>
<form method="post" action="<?= eAdmin(APP_BASE . '/logout.php') ?>">
    <input type="hidden" name="csrf_token" value="<?= eAdmin(csrfToken()) ?>">
    <button type="submit">Logout</button>
</form>
```

Untuk UX, bungkus tautan Tambah, tautan Edit, dan form Hapus dengan kondisi berikut; jangan membungkus seluruh kolom tabel sehingga jumlah sel berubah tanpa menyesuaikan header:

```
<?php if (in_array($penggunaAktif['role'], ['admin', 'operator'], true)): ?>
    <!-- Letakkan kontrol tambah/edit/hapus yang sudah ada di sini. -->
<?php endif; ?>
```

Potongan di atas adalah pola pembungkus, bukan pengganti tombol. Pertahanan utama tetap `requireRole()` pada server. Jika tombol belum disembunyikan, viewer akan mendapatkan 403 ketika menekannya, bukan memperoleh akses tulis.

10. Melindungi halaman lama Modul 4

Panel baru terlindungi tidak cukup. Halaman `tambah.php`, `edit.php`, dan `hapus.php` lama masih dapat mengubah database jika tidak diberi guard.

10.1 Halaman baca: `index.php` dan `detail.php` di root

Tambahkan setelah `<?php` dan setelah `declare` jika ada, **sebelum koneksi/SQL/output yang sudah ada**:

```
require_once __DIR__ . '/auth.php';
$penggunaAktif = requireLogin();
```

Ubah pemuatan koneksi lama menjadi `require_once` jika belum demikian. Helper Modul 4 menggunakan nama `e()`, sedangkan helper panel memakai `eAdmin()`, sehingga tidak perlu mengganti seluruh template lama.

10.2 Halaman tulis: `tambah.php` dan `edit.php`

Tambahkan di awal sebelum pemrosesan data:

```
require_once __DIR__ . '/auth.php';
$penggunaAktif = requireRole(['admin', 'operator']);
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    verifyCsrf();
}
```

Pada form bersama `form_siswa.php`, tambahkan **di dalam elemen** `<form method="post">`:

```
<input type="hidden" name="csrf_token" value="<?= eAdmin(csrfToken()) ?>">
```

Form bersama tersebut digunakan tambah dan edit, sehingga satu penambahan mencakup keduanya. Token harus dikirim pada POST yang benar, bukan form pencarian GET.

10.3 Endpoint `hapus.php`

Tambahkan sebelum menjalankan logika penghapusan yang sudah ada:

```
require_once __DIR__ . '/auth.php';
$penggunaAktif = requireRole(['admin', 'operator']);
if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
    header('Allow: POST');
    http_response_code(405);
    exit('Gunakan POST.');
}
verifyCsrf();
```

Tambahkan hidden `csrf_token` yang sama pada setiap form hapus di `index.php` lama. Jangan mengganti field ID lama; cukup tambahkan token. Jika ada form hapus lain, semuanya harus diperbarui.

10.4 Inventaris jalur akses

Lokasi	Proteksi yang wajib
Root `index.php`, `detail.php`	Login sebelum query/output

Lokasi	Proteksi yang wajib
Root `tambah.php`, `edit.php`	Login + role; CSRF pada POST
Root `hapus.php`	Login + role + POST + CSRF
`admin/index.php`	Bootstrap login; role pada form/simpan/hapus
Endpoint ekspor/API tambahan	Guard sendiri, bukan mengandalkan menu
`login.php`	Publik, POST login tetap CSRF
`logout.php`	POST + CSRF
`buat_akun.php`	Menolak akses web, hanya CLI

Hapus `tes_koneksi.php` setelah pengujian koneksi selesai karena tidak diperlukan sebagai halaman publik. Helper dan repository tidak boleh memiliki aksi perubahan data saat dibuka langsung. Untuk produksi, pindahkan konfigurasi/kelas nonpublik ke luar document root bila arsitektur server memungkinkan.

Jangan hanya menaruh guard di `header.php`: proses penyimpanan dapat berjalan sebelum header dimuat, dan endpoint tanpa template tidak akan terlindungi.

11. Menguji session dan hak akses

Gunakan profil browser atau jendela privat terpisah untuk membandingkan akun. Jangan mengandalkan dua tab biasa untuk dua identitas berbeda karena cookie session-nya sama.

No.	Skenario	Hasil
1	Buka panel tanpa login	Redirect 303 ke login
2	Login salah	HTTP 401, pesan umum, tidak ada identitas login
3	Login benar	Redirect panel; ID session berubah
4	Viewer membuka daftar/rapor	Boleh
5	Viewer membuka URL form langsung	HTTP 403
6	Viewer POST simpan/hapus langsung	HTTP 403, database tetap
7	Operator/admin CRUD dengan token sah	Berhasil sesuai validator
8	POST tanpa token atau token salah	HTTP 403
9	GET logout	HTTP 405; session tidak dihancurkan
10	POST logout sah	Cookie dihapus dan session dihancurkan
11	Request panel setelah logout	Redirect login
12	Buka halaman lama tanpa login	Redirect login juga
13	Nonaktifkan user di database	Request berikutnya meminta login

No.	Skenario	Hasil
14	Ubah operator menjadi viewer	Request berikutnya tidak boleh menulis
15	Akses `buat_akun.php` dari browser	HTTP 404, tidak membuat akun
16	Token sebelum login dipakai sesudah login	Ditolak karena token dirotasi

11.1 Menguji timeout tanpa menunggu lama

Di lingkungan latihan, ubah sementara `AUTH_IDLE_SECONDS` menjadi 20 dan `AUTH_ABSOLUTE_SECONDS` menjadi 60.

1. Login dan tunggu lebih dari 20 detik tanpa request.
2. Muat ulang; harus kembali ke login.
3. Login lagi dan kirim request setiap 10 detik.
4. Setelah lebih dari 60 detik, tetap harus login ulang karena absolute timeout.
5. Kembalikan konfigurasi normal setelah pengujian.

Idle timeout memakai aktivitas request server, bukan gerakan mouse. Polling otomatis dapat memperpanjang idle timeout, tetapi tidak absolute timeout.

11.2 Pengujian perubahan akun

Gunakan username akun latihan, bukan akun utama:

```
UPDATE users SET aktif = 0 WHERE username = 'viewer_latihan';
UPDATE users SET aktif = 1 WHERE username = 'viewer_latihan';
UPDATE users SET role = 'viewer' WHERE username = 'operator_latihan';
```

Periksa jumlah baris yang berubah sebelum melanjutkan. Akun yang diaktifkan kembali perlu login ulang jika session sebelumnya sudah dibersihkan.

12. Brute force dan batas implementasi lokal

Kode login inti **belum menyediakan rate limiting**. Pesan umum, CSRF, hashing, dan session timeout tidak mencegah percobaan password dalam jumlah besar.

Sebelum dipublikasikan, rancang dan uji:

1. Pembatasan percobaan berdasarkan identitas akun serta sumber request.
2. Penyimpanan counter bersama, misalnya database atau Redis, bukan hanya session browser.
3. Pembaruan counter atomik agar request paralel tidak melewati batas.
4. Waktu tunggu bertahap dan respons 429 sesuai kebijakan.
5. Pemulihan otomatis agar penyerang tidak mudah mengunci akun orang lain selamanya.
6. Kebijakan proxy/IP tepercaya agar alamat sumber tidak dipalsukan lewat header.
7. Log kejadian tanpa password atau token.

`sleep()` pada setiap kegagalan bukan pengganti rate limiting dan dapat menghabiskan worker. Pembatasan hanya berdasarkan cookie session mudah dihindari dengan membuat session baru. Tidak ada satu angka batas

yang cocok untuk seluruh sekolah; tentukan berdasarkan kebutuhan dan kapasitas.

13. Fitur lanjutan: desain sebelum implementasi

13.1 Ganti password

Alur yang disarankan:

- Wajib login dan verifikasi password lama.
- POST + CSRF.
- Validasi password baru dan konfirmasi.
- Hash baru dengan `password_hash()`.
- Rotasi session saat ini; sediakan cara mencabut session lain.

Implementasi saat ini tidak mencabut semua session hanya karena hash berubah. Untuk itu tambahkan versi autentikasi pada tabel user dan session, atau penyimpanan session per akun yang dapat dicabut.

13.2 Reset password

Jangan mengirim password asli atau password baru melalui URL. Gunakan token acak sekali pakai, simpan hash token, batasi umur, batalkan token setelah dipakai, dan gunakan respons umum agar keberadaan akun tidak mudah ditebak. Pengiriman email serta rate limiting harus dirancang sebagai satu alur.

13.3 MFA dan reauthentication

Tindakan sensitif seperti mengganti role atau mengekspor seluruh data pribadi dapat memerlukan autentikasi ulang dan MFA. MFA bukan sekadar menambah kolom kode tetap. Gunakan implementasi dan library yang terpelihara serta prosedur recovery yang aman.

13.4 Audit trail

Catat siapa, aksi apa, objek mana, waktu, dan hasil. Batasi akses log. Jangan menyimpan password, hash password, cookie session, atau seluruh POST secara membabi buta. Untuk perubahan nilai, tentukan kebijakan pencatatan nilai lama dan baru sesuai kebutuhan serta privasi sekolah.

14. Latihan bertingkat dan pembahasan

Latihan A — matriks akses

Buat tiga akun CLI. Uji seluruh endpoint lama dan baru dengan ketiga role. Lampirkan status HTTP dan bukti bahwa database tidak berubah pada penolakan.

Pembahasan: pengujian tombol saja tidak cukup. Akses URL langsung dan POST langsung merupakan pengujian wajib. Gunakan DevTools Network untuk melihat metode dan status.

Latihan B — dashboard sesuai role

Tampilkan menu tulis hanya kepada admin/operator. Tetap pertahankan pemeriksaan role server.

Pembahasan: UI mengikuti kebijakan, tetapi keputusan akhir berada di server. Jika role berubah saat halaman masih terbuka, submit berikutnya tetap harus ditolak.

Latihan C — endpoint JSON terautentikasi

Buat endpoint baca jumlah siswa. Jika belum login, kembalikan JSON berstatus 401, bukan redirect HTML. Jika sudah login, kirim JSON berisi total. Gunakan `Content-Type: application/json` dan `Cache-Control: no-store`.

Pembahasan: gunakan `currentUser()` untuk memutuskan respons API, lalu `json_encode()`. Jangan memakai `requireLogin()` apa adanya jika kontrak API mengharuskan JSON karena helper itu dirancang untuk halaman browser.

Latihan D — desain rate limit

Tulis skenario counter atomik, batas waktu, reset setelah login berhasil, serta pengujian request serentak. Implementasi hanya dianggap selesai jika dapat dibuktikan tidak dapat dilewati dengan mengganti cookie session.

15. Troubleshooting

Gejala	Penyebab umum	Tindakan
Redirect login terus-menerus	Login memuat bootstrap terlindungi	Login hanya memuat `auth.php`
Session hilang setiap request	Cookie path/base salah atau Secure pada HTTP	Cocokkan APP_BASE dan konfigurasi HTTPS
`headers already sent`	HTML, BOM, atau spasi sebelum bootstrap	Pindahkan auth sebelum output
Viewer masih bisa menghapus	Endpoint lama belum diberi guard	Audit seluruh jalur mutasi
CSRF gagal setelah login	Form lama masih menggunakan token sebelum rotasi	Muat ulang form
Password benar tetapi ditolak	Akun nonaktif, username berbeda, password pernah di-trim	Periksa kebijakan dan status akun
Cookie tidak terhapus	Path/domain penghapusan berbeda	Gunakan parameter session yang sama
Login berhasil tetapi database error	Tabel/kolom users belum sesuai	Periksa migrasi dan log server
Dua tab memakai akun yang sama	Tab berbagi cookie	Gunakan profil/jendela privat berbeda

16. Rubrik penilaian

Aspek	Bobot
Login/logout dan hash password	25

Aspek	Bobot
Otorisasi seluruh endpoint lama dan baru	30
Session, timeout, cookie, CSRF	25
Pengujian negatif dan dokumentasi	15
Penjelasan batas produksi	5

Kriteria wajib: viewer tidak dapat mengubah data melalui endpoint mana pun, password tidak tersimpan sebagai teks biasa, serta semua request perubahan memeriksa CSRF.

17. Daftar periksa akhir

- [ ] Tabel pengguna terpisah dari siswa.
- [ ] Akun pertama dibuat lewat CLI, tanpa password hardcoded.
- [ ] Login memakai prepared statement dan `password_verify()`.
- [ ] ID session dan token dirotasi setelah login.
- [ ] Role/status aktif dibaca ulang dari database.
- [ ] Timeout idle dan absolute sudah diuji.
- [ ] Logout POST menghapus cookie dan session.
- [ ] Halaman lama Modul 4 ikut diproteksi.
- [ ] Tidak ada endpoint ekspor/API yang mengandalkan menu sebagai perlindungan.
- [ ] Tidak mengklaim aplikasi siap produksi sebelum rate limit dan deployment diperiksa.

18. Referensi

- PHP password_hash
- PHP password_verify
- PHP session security
- OWASP Authentication Cheat Sheet
- OWASP Session Management Cheat Sheet
- OWASP CSRF Prevention Cheat Sheet